

Guia de trabajos prácticos N° 5

Subrutinas, strings y traducción de C a assembler

1. Hacer una subrutina que calcule la potencia de dos números pasados como parámetro. Resolver en forma iterativa y recursiva.
2. Armar una “librería” de funciones que permitan trabajar con string con las siguientes subrutinas:
 - a. **SLEN**: recibe un puntero a un string y devuelve en ECX la cantidad de caracteres (sin incluir el terminator) del string.
 - b. **SCPY**: recibe dos punteros y permite copiar una cadena de caracteres de una posición de memoria a otra.
 - c. **SCMP**: recibe dos punteros a strings, y resta carácter a carácter (sin alterar los strings) hasta que exista una diferencia devolviéndola en EAX o 0 si los strings son iguales.
 - d. **SCAT**: recibe dos punteros a strings y concatena al primero el segundo (es responsabilidad del programador que a continuación del primer string no se pisen datos).
 - e. **SPLIT**: recibe un puntero a string, un carácter, y un puntero a un array de punteros. Divide el strings en varios strings reemplazando el carácter por el terminator y completa el array de punteros con el puntero al primer carácter de cada string, utilizando -1 para marcar el fin.
 - f. **STRIM**: recibe un puntero a string como parámetro variable, y devuelve el string quitando los “espacios” (white spaces) del comienzo y del final.
3. Realizar subrutinas para convertir:
 - a. Un string, que representa un número entero en base 10, a un número.
 - b. Un número entero a su representación en string.
4. Realizar un programa *op.vmx* para interpretar una operación matemática con dos operandos, recibida por parámetros, y que se muestre el resultado por pantalla. Ejemplo:

```
> vmx op.vmx -p 45 + 20
= 65
```

Los operadores admitidos son: suma (+), resta (-), multiplicación (*) y división (/)

5. Interpretar una operación matemática escrita en notación polaca inversa y calcular el resultado.

Notación aritmética: $(20 + 3) * 6$

La misma operación escrita en notación polaca inversa: $20\ 3\ +\ 6\ *$

La ventaja de este tipo de notación es que no es necesario utilizar paréntesis y sus operaciones pueden ser resueltas utilizando una pila.

Ejemplo: entrada: “20 3 + 6 *” salida: “138”

6. Solicitar una frase ingresada por el usuario y determinar si es palíndroma, sin tomar en cuenta mayúsculas, minúsculas, espacios, signos de puntuación o acentos. Por ejemplo: si ingresa “La ruta nos aportó otro paso natural.” se debe escribir “Es frase palíndroma”.

7. Indicar en los siguientes casos de invocación a funciones, cómo sería la invocación a la subrutina en assembler. Considerar que las variables a, b y c se encuentran en las posiciones de memoria BP-4, BP-8 y BP-12, respectivamente, y corresponden a variables locales de la función que invoca.

- `proce(a, b, c);`
- `c = func(*a, b);`
- `proce(&a, *b, *c);`
- `*c = func(a + b);`

8. Traducir a assembler los siguientes programas e indicar cómo quedaría la pila al final de sus ejecuciones:

a. `int j=3;`

```
void main(){
    int arr[5];
    arr[0] = 1020;
    arr[1] = arr[0] | 0x3FF;
    calculo(j, arr);
}

void calculo( int x, int vec[]){
    vec[x] = vec[x-1] & vec[0];
}
```

b. `int vec[9];`

```
void main(){
    int i = 2;
    vec[i] = 501;
    vec[0] = 0xF0F0 & vec[i];
    proce(i, vec);
}

void proce( int x, int w[]){
    w[x-1] = w[x] | w[0];
}
```

9. Traducir a assembler los siguientes programas:

a. `void main() {`
`int* vec = malloc(sizeof(int) * 100);`
`short int* z;`
`z = malloc(sizeof(short int));`
`*z = 100;`
`func(vec, *z);`
`}`
`int sum(int vec[], short int* n) {`
`static int s = 0;`
`register int i;`
`for (i = 0; i < *n; i++)`
 `s = s + vec[i];`
`return s;`
`}`

b. `#typedef int op(short int, short int);`
`op ops[] = {add, sub, mul, div};`
`char* nom[] = {"ADD", "SUB", "MUL", "DIV"};`
`void main() {`
`short int x, y;`
`scanf("%d %d", &x, &y);`
`for (int i = 0; i < 4; i++) {`
 `printf("%s =", nom[i]);`
 `printf("%d\n", ops[i](a, b));`
`}`

```
int func(int vec[], short int z) {
    int s;
    short int i;
    vec[0] = 1;
    for (i = 1; i < z; i++)
        vec[i] = vec[i-1] * i;
    s = sum(vec, &z);
    print_int(s);
}
```

```
int add(short int a, short int b) {
    return a + b;
}

int sub(short int a, short int b) {
    return a - b;
}

int mul(short int a, short int b) {
    return a * b;
}

int div(short int a, short int b) {
    return a / b;
}
```